

KamaChat Server 模拟面试 Q&A

这是一份基于简历中 KamaChat Server 项目的模拟面试 Q&A。作为面试官，我将从**系统架构、高并发处理、消息可靠性、缓存策略、安全与 AI 业务场景**等多个维度对你进行考察。

一、架构设计与消息队列 (Architecture & MQ)

Q1: 你的项目中使用了 Kafka 作为消息队列，为什么要引入 Kafka？如果在单机环境下，使用 Channel 也能解耦，接入 Kafka 解决了什么痛点？

A: 引入 Kafka 主要是为了解决**系统的水平扩展性**和**高并发下的消息吞吐瓶颈**。虽然在单机模式下 Go 的 Channel 非常轻量且高效，但作为一个即时通讯服务端，单机连接数和内存终究有限。一旦需要部署多台 WebSocket 网关节点来承载海量用户的长连接时，使用单机 Channel 就会导致不同节点之间的消息无法互通。接入 Kafka 后：

- 业务解耦与异步削峰：**用户的发消息请求（HTTP 或 WS）到达后端后，直接存入 Kafka，快速给客户端返回 Ack，避免了同步写库带来的阻塞请求，大大提升了接口的吞吐量。
- 分布式路由：**Kafka 作为一个中心化的消息总线，各个 WebSocket 节点作为消费者订阅消息，可以轻松实现分布式环境下的消息投递。
- 数据可靠性：**相比于 Channel 在进程重启时会丢失数据，Kafka 提供了持久化机制，保证了聊天记录的不丢失。

Q2: 在 IM 系统中，如何保证消息的顺序性？如果使用 Kafka，你是如何设计 Partition Key 的？

A: IM 系统中，同一个会话（单聊或群聊）的消息必须是有序的。如果不做控制，Kafka 的多 Partition 消费可能会导致消息乱序。我的解决方案是：

- 分配 Partition Key:** 在将消息写入 Kafka Publish 时，我通过特定的逻辑生成一个逻辑会话 Key (Logical Session Key) 作为 Kafka 的 Partition Key。例如，单聊场景下，我将 SenderID 和 ReceiverID 排序后拼接 ($\min(A, B)_{\max(A, B)}$)，群聊则直接使用 GroupID。
 - 局部有序机制：**因为 Kafka 保证了同一个 Key 的消息会被路由到同一个 Partition，而同一个 Partition 的消息是有序的。这样就确保了**任意一个特定会话内的消息处理是绝对有序的**，而不同会话之间的消息可以继续并发处理，兼顾了顺序性和整体的高并发性能。
-

二、WebSocket 与实时通信 (WebSocket & Real-time)

Q3: 你的 WebSocket 网关是如何管理海量用户连接的？如果一个用户在多端（如 PC 端和 Web 端）同时登录，系统如何处理消息推送？

A: WebSocket 的连接管理依赖于一个全局的 ClientManager（或类似的 Session 管理器）。

- 连接存储：**当用户建立 WS 连接并鉴权通过后，我会将该用户的连接对象保存在内存的 Map 中。为了支持多端登录，这个 Map 的结构通常是 `map[UserID]map[DeviceID]*WebSocketConn`。
 - 多端推送：**当消费到发给某位用户的消息时，服务端会通过 UserID 查找该表。如果匹配到多个连接实例（即该用户同时在线多个端），则会使用 Go 的 goroutine 并发地向这些端循环写入消息数据。
 - 心跳保活：**由于网络可能波动或客户端异常断线，服务端和客户端之间实现了 Ping/Pong 心跳机制，定时清理掉已经失效的死连接（Dead Connections），防止内存泄漏和文件描述符被打满。
-

三、缓存设计与性能优化 (Cache & Performance)

Q5: 项目中用到了 Redis 作为缓存, 能详细说说缓存策略吗? 你们是如何避免缓存击穿和缓存雪崩的?

A: 即时通讯系统读多写少的情况很常见, 比如获取群成员列表、获取用户个人信息等。

1. **缓存策略 (Cache Aside):** 采用标准的旁路缓存模式。先读 Redis, 未命中再读 MySQL 并回写 Redis。数据更新时, 由于聊天一致性要求, 通常采用先修改数据库, 再删除缓存的方式。
2. ****防缓存击穿:** 引入 `singleflight`。在我项目的 `internal/dao/redis/cache/` 目录下, 我使用了 Go 官方的 `golang.org/x/sync/singleflight`。当某个热点键过期时, 大量并发请求只会有一个去穿透查询 MySQL, 其他请求会阻塞等待第一个请求的结果, 极大保护了数据库。
3. **防缓存雪崩:** 核心策略是**打散过期时间**。对批量设置的缓存 (如活跃的用户信息), 在基础过期时间上加上一段随机抖动值 (Random Jitter), 防止大量 Key 在同一时刻集体失效。

四、安全与鉴权设计 (Security & Auth)

Q6: 你的系统使用了 JWT 双 Token 机制 (Access Token + Refresh Token), 请解释一下为什么这样做? 以及 Refresh Token 被盗了怎么办?

A:

1. **双 Token 目的:** 为了安全和用户体验的平衡。Access Token 的过期时间很短 (例如 15 分钟), 即使被劫持, 攻击窗口也很小。由于 Access Token 频繁过期, 如果让用户频繁重新登录体验非常差。因此引入有效期较长 (如 7 天) 的 Refresh Token, 当短 Token 过期时, 客户端自动携带长 Token 重新换发新的短 Token, 实现**无感刷新**。
2. **安全性兜底:** 如果 Refresh Token 被盗, 这确实有风险。因此在设计上:
 - 当更换密码、主动退出登录、或系统检测到异常设备登录时, 会在服务端的 Redis 内将当前的 Refresh Token 拉黑 (黑名单机制) 或使该用户的 Token Version 升代。
 - Refresh Token 服务端只验证一次或与设备指纹 (DeviceFingerprint) 绑定, 防止被跨设备重放攻击。

Q7: 项目介绍中提到了 IDOR (水平越权) 防护, 你在具体业务中是如何防范的? 举个例子。

A: 在 IM 系统中非常容易出现水平越权, 比如用户 A 将请求参数里的 `target_id` 改成用户 B 的, 尝试获取 C 给 B 的私聊记录。我的防护方案是:**强依赖服务端 Session 鉴权**。所有的 API 请求中, 只要涉及到操作实体 (发消息、查记录), 对应的 `owner_id` 或 `user_id` 绝对不能从前端的 HTTP Body 或 Query 中获取, 而是必须通过 API 网关的鉴权中间件解析 JWT Token, 从中提取出真实的 UserID 并注入到 Gin 的上下文 (`c.Set("userID", uid)`) 中。在业务层, 直接校验 上下文获取的真实UID 是否具有访问当前资源ID(如群组记录) 的权限。
